

SecurAccess

Biometric Access Control System via Facial Recognition and Digital Watermarking

Biometrics and Digital Watermarking Project

Academic Year 2025–2026

Group : Group_X
Members : Member 1 Member 2 Member 3
Supervisor : Prof. [Supervisor's Name]
Program : [Program Name / Master]
Date : May 2026



Figure 1: University Logo

Filename : Group_X_Project_Y.pdf

Contents

1	Introduction	4
1.1	Project Context	4
1.2	Objectives	4
1.3	Technologies Used	5
2	Technical Description	6
2.1	Global Architecture	6
2.1.1	Architecture Diagram	7
2.1.2	Project Structure	7
2.2	Database Schema	8
2.3	Navigation System	9
2.4	Session Management	9
2.5	Hierarchical Role System	9
3	Algorithms and Technical Justifications	11
3.1	Authentication Pipeline	11
3.2	Face Detection Haar Cascade	12
3.2.1	Principle	12
3.2.2	Chosen Parameters	13
3.2.3	Preprocessing	13
3.2.4	Choice Justification	13
3.3	Facial Recognition ArcFace (InsightFace)	13
3.3.1	Principle	13
3.3.2	Model Used	14
3.3.3	Embedding Generation Process	14
3.3.4	Matching Euclidean Distance	14
3.3.5	Decision Thresholds	14
3.4	Digital Watermarking HMAC-SHA256	15
3.4.1	Principle	15
3.4.2	Payload Construction	15
3.4.3	Integrity Verification	15
3.4.4	Choice Justification	16

4	Performance Analysis	17
4.1	Experimental Protocol	17
4.1.1	Test Environment	17
4.1.2	Dataset	17
4.2	Biometric Metrics	18
4.2.1	Definitions	18
4.2.2	Formulas	18
4.2.3	Experimental Results	18
4.3	Response Time	20
4.4	Interpretation of Results	20
5	Attack Simulations	22
5.1	Objective	22
5.2	Attack 1 Spoofing by Printed Photo	22
5.2.1	Description	22
5.2.2	Protocol	22
5.2.3	Results	23
5.3	Attack 2 Spoofing by Screen (Replay Attack)	23
5.3.1	Description	23
5.3.2	Protocol	24
5.3.3	Results	24
5.4	Attack 3 Image Noise	26
5.4.1	Description	26
5.4.2	Protocol	26
5.4.3	Results	26
5.5	Attack 4 JPEG Compression	27
5.5.1	Description	27
5.5.2	Results	27
5.6	Attack 5 Log Alteration (Watermarking)	27
5.6.1	Description	27
5.6.2	Protocol	27
5.6.3	Results	28
5.7	Robustness Summary	29
6	User Interface	30
6.1	Visual Theme	30
6.2	Application Pages	30
6.2.1	Login Page	30
6.2.2	Welcome Page	31
6.2.3	Access Denied Page	33
6.2.4	Unknown Face Page	34
6.2.5	Enrollment Page	35
6.3	Administrator Dashboard	36

6.4	Proposed Interface Improvements	39
7	Conclusion	40
7.1	Project Summary	40
7.2	Identified Limitations	40
7.3	Future Improvements	41

Chapter 1

Introduction

1.1 Project Context

Biometric security today represents a major challenge in access control systems. Faced with the limitations of traditional methods (passwords, badges, PIN codes), facial recognition is establishing itself as a solution that is both ergonomic and secure.

The **SecurAccess** project is part of the *Biometrics and Digital Watermarking* module. It proposes a complete access control system based on facial recognition, integrating a digital watermarking mechanism to ensure the integrity of authentication logs.

1.2 Objectives

The main objectives of the project are:

1. **Biometric Authentication** Identify users via their faces in real-time using a webcam.
2. **Role Management** Implement a hierarchical role system (Admin, Ultimate, Pro, User, Unauthorized) with granular access control.
3. **Digital Watermarking of Logs** Ensure the integrity of authentication logs via HMAC-SHA256 signatures.
4. **Administration Dashboard** Provide a comprehensive dashboard for managing users, logs, alerts, and registration requests.
5. **Attack Detection** Analyze the system's robustness against spoofing, noise, and compression attacks.

1.3 Technologies Used

Technology	Role
Python 3.11	Main development language
PyQt5 5.15	Desktop graphical interface (native widgets)
OpenCV 4.9	Webcam capture and face detection (Haar Cascade)
InsightFace 0.7.3	Facial embedding extraction via ArcFace model
ONNX Runtime	ArcFace model inference on CPU
SQLite	Local database for users, logs, and alerts
HMAC-SHA256	Digital watermarking for log integrity

Table 1.1: Technology stack of the SecurAccess project

Chapter 2

Technical Description

2.1 Global Architecture

SecurAccess adopts a modular **MVC** (Model–View–Controller) architecture, structured into distinct layers:

Layered Architecture

- **Presentation Layer (UI)** PyQt5 Pages: Login, Welcome, Admin Dashboard, Access Denied, Unknown Face, Enrollment
- **Controller Layer** `AuthController` orchestrates the authentication pipeline
- **Service Layer** Business logic: Auth, Camera, FaceDetection, FaceBiometric, Log, Watermark, Alert, Email, Enrollment, Upgrade
- **Data Layer** SQLite via `DatabaseService`, `dataclass` models

2.1.1 Architecture Diagram

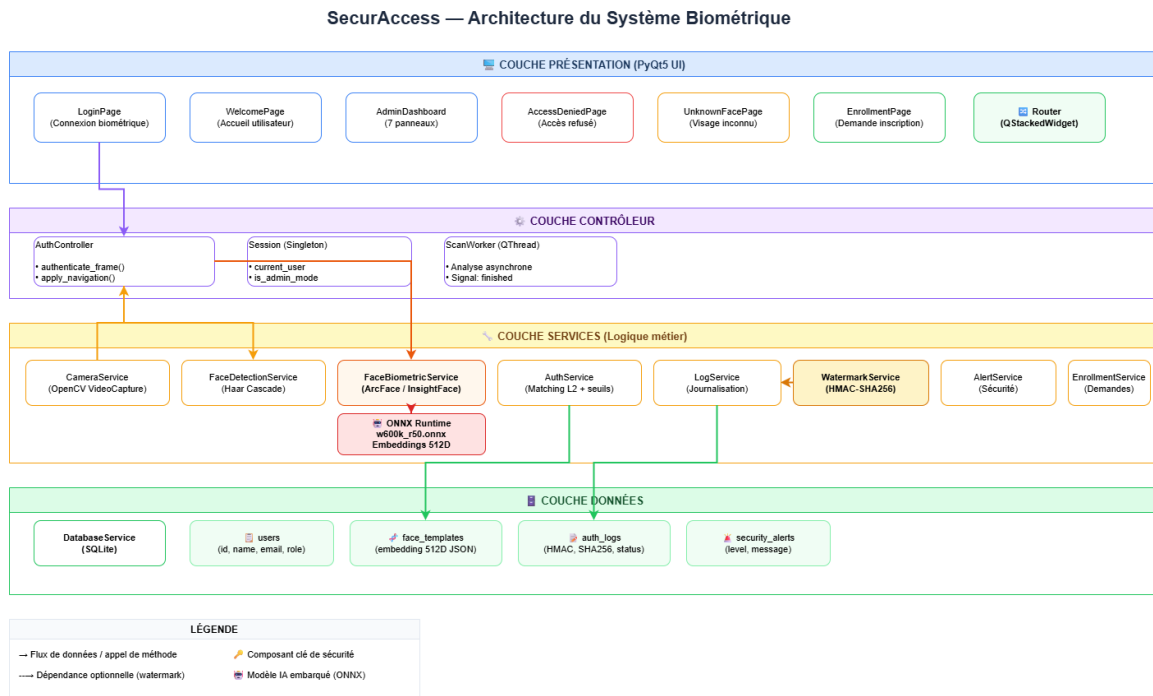


Figure 2.1: Global architecture diagram of the SecurAccess system

2.1.2 Project Structure

```

1 SecurAccess/
2 |-- main.py                                # Entry point
3 |-- data/
4 |   |-- securaccess.db                    # SQLite Database
5 |   |-- faces/                           # Enrolled face images
6 |-- app/
7 |   |-- core/
8 |   |   |-- router.py                     # Navigation (QStackedWidget)
9 |   |   |-- session.py                    # User session (Singleton)
10 |   |-- controllers/
11 |   |   |-- auth_controller.py            # Authentication pipeline
12 |   |-- models/
13 |   |   |-- user.py                      # User model + roles
14 |   |   |-- auth_result.py               # Authentication result
15 |   |   |-- face_detection_result.py
16 |   |   |-- enrollment_request.py
17 |   |-- services/
18 |   |   |-- auth_service.py               # Facial authentication
19 |   |   |-- face_detection_service.py     # Haar Cascade detection
20 |   |   |-- face_biometric_service.py    # ArcFace embeddings
21 |   |   |-- camera_service.py             # Webcam management

```



```

22 |   |   |-- database_service.py      # SQLite access
23 |   |   |-- watermark_service.py    # HMAC-SHA256 watermarking
24 |   |   |-- log_service.py          # Watermarked logging
25 |   |   |-- alert_service.py        # Security alerts
26 |   |   |-- email_alert_service.py  # Email notifications
27 |   |   |-- enrollment_service.py   # Enrollment requests
28 |   |   |-- upgrade_service.py      # Upgrade requests
29 |   |-- ui/
30 |   |   |-- login_page.py           # Login page
31 |   |   |-- welcome_page.py        # User home page
32 |   |   |-- access_denied_page.py   # Access denied
33 |   |   |-- unknown_face_page.py    # Unknown face
34 |   |   |-- enrollment_page.py      # Enrollment form
35 |   |   |-- admin/                  # Admin dashboard (7 panels)
36 |   |   |-- components/             # Reusable components
37 |   |   |-- dialogs/                # Modal dialogs
38 |   |-- resources/
39 |       |-- styles.py               # Theme and color palette
40 |-- scripts/
41     |-- enroll_admin.py             # Webcam enrollment script

```

Listing 2.1: Project Tree

2.2 Database Schema

The system uses **SQLite** with four main tables:

Table	Primary Key	Description
users	id (AUTO)	Registered users (face_id, name, email, role, status)
face_templates	id (AUTO)	512D ArcFace embeddings stored in JSON, linked to users.id
auth_logs	id (AUTO)	Authentication logs with HMAC watermark and SHA256 hash
security_alerts	id (AUTO)	Security alerts by level (critical, warning, info)

Table 2.1: SQLite database tables

Database Schema

```

erDiagram
    USERS ||--o{ FACE_TEMPLATES : "owns"
    USERS ||--o{ AUTH_LOGS : "creates"
    SECURITY_ALERTS }o--|| USERS : "concerns"

    USERS {
        int id PK
        string face_id UK
        string full_name
        string email
        string role
        int is_active
    }

    FACE_TEMPLATES {
        int id PK
        int user_id FK
        string embedding
        string image_path
        string created_at
    }

    AUTH_LOGS {
        int id PK
        string user_name
        string user_role
        string status
        float confidence
        string details
        string watermark
        string payload_hash
        int integrity_ok
        string created_at
    }

    SECURITY_ALERTS {
        int id PK
        string level
        string title
        string message
        int is_read
        string created_at
    }

```

Figure 2.2: Database relational schema

2.3 Navigation System

Navigation is managed by a **Router** (Singleton pattern) which controls a `QStackedWidget`. All pages are instantiated once at startup and stacked. Navigation is done by name:

```

1 Router.get_instance().navigate("welcome") # Displays the welcome
   page
2 Router.get_instance().go_back()           # Go back

```

Listing 2.2: Navigation example

Registered pages: login, welcome, access_denied, unknown_face, enrollment, admin_dashboard.

2.4 Session Management

The user session is a **Singleton** (`Session`) that stores the logged-in user and their mode (admin or standard user). It is accessed by all pages to adapt the display and permissions.

2.5 Hierarchical Role System

The system implements five role levels with a numerical hierarchy:

Role	Level	Access
Admin	100	Full access + Administration dashboard
Ultimate	50	All user sections
Pro	30	Pro + User sections
User	10	Basic sections only
Unauthorized	0	Recognized but access revoked

Table 2.2: Role hierarchy in SecurAccess

Chapter 3

Algorithms and Technical Justifications

3.1 Authentication Pipeline

The authentication flow follows a sequential pipeline in three steps:

1. **Capture** The webcam captures a 640×480 BGR frame via OpenCV (`CameraService`).
2. **Detection** The Haar Cascade classifier locates faces in the frame.
3. **Recognition** The ArcFace embedding of the facial ROI is compared to stored templates.

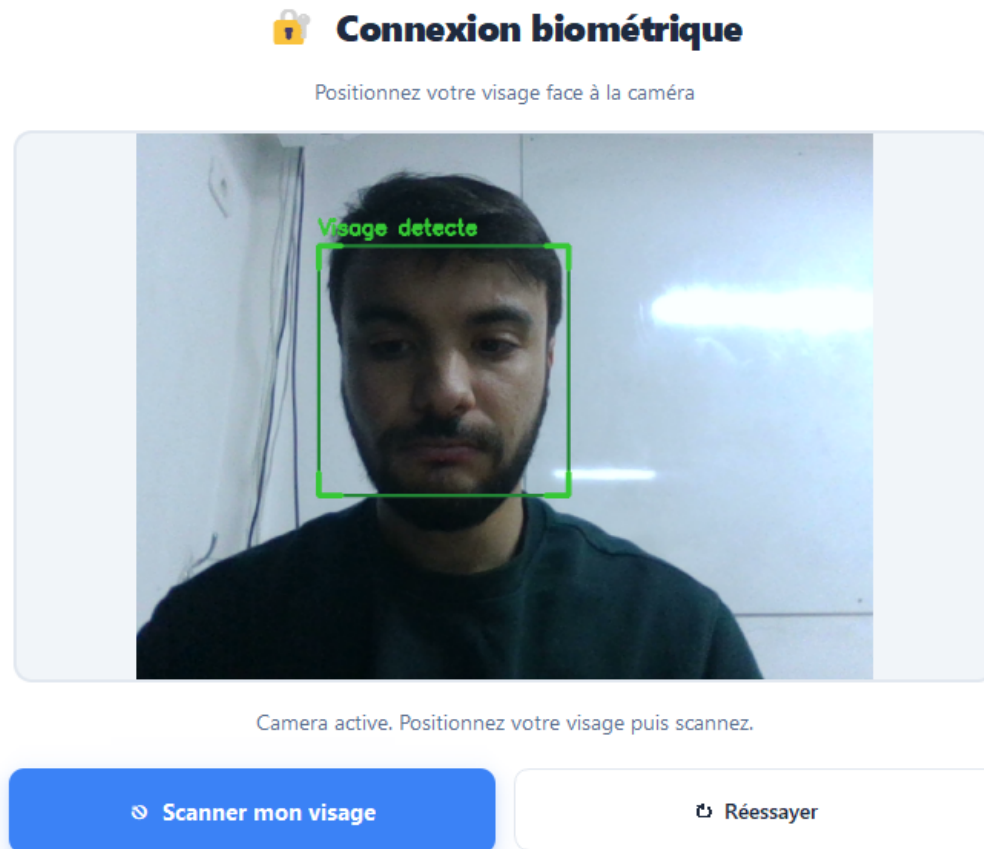


Figure 3.1: Complete facial authentication pipeline

3.2 Face Detection Haar Cascade

3.2.1 Principle

Detection uses the Viola-Jones cascade classifier (**Haar Cascade**) provided by OpenCV. This detector scans the image at multiple scales applying Haar features to identify facial characteristics (eyes, nose, mouth).

3.2.2 Chosen Parameters

Parameter	Value	Justification
<code>scaleFactor</code>	1.2	Trade-off between speed and sensitivity. A smaller value (1.05) would be more accurate but slower.
<code>minNeighbors</code>	6	Filters false positives: a face must be detected by at least 6 neighboring windows.
<code>minSize</code>	(80, 80)	Ignores faces that are too small (distant or artifacts).

Table 3.1: Haar Cascade detector configuration

3.2.3 Preprocessing

Before detection, two operations are applied:

1. **Grayscale Conversion** `cv2.cvtColor(frame, COLOR_BGR2GRAY)`
2. **Histogram Equalization** `cv2.equalizeHist(gray)` to normalize contrast and improve robustness to lighting variations.

3.2.4 Choice Justification

Why Haar Cascade?

- **Speed** Real-time execution on CPU, suitable for 30 FPS refresh rate.
- **Reliability** Proven model since 2001, robust under normal lighting conditions.
- **Lightweight** No GPU dependency, runs on any machine.
- **Complementarity** Fine-grained recognition is delegated to ArcFace in the second step.

3.3 Facial Recognition ArcFace (InsightFace)

3.3.1 Principle

ArcFace is a facial recognition model based on a deep convolutional neural network (**ResNet-50**). It transforms a 112×112 face image into a **512-dimensional embedding vector**, normalized in L2 norm.

The uniqueness of ArcFace lies in its loss function (*Additive Angular Margin Loss*) which maximizes the angular margin between classes, producing highly discriminative embeddings.

3.3.2 Model Used

Property	Value
Model	w600k_r50.onnx (buffalo_1 pack)
Architecture	ResNet-50
Input Size	112 × 112 pixels (BGR)
Embedding Size	512 dimensions (float32)
Normalization	L2 (unit vector)
Runtime	ONNX Runtime (CPU, ctx_id=-1)
Model Size	~280 MB (downloaded once)

Table 3.2: Characteristics of the ArcFace model

3.3.3 Embedding Generation Process

```

1 def build_embedding(self, face_roi: np.ndarray) -> np.ndarray:
2     # 1. Resize to 112x112
3     resized = cv2.resize(face_roi, (112, 112)).astype(np.uint8)
4     # 2. Extract features via ArcFace model
5     raw_feat = model.get_feat(resized)
6     # 3. Flatten and convert to float32
7     vector = np.array(raw_feat).flatten().astype(np.float32)
8     # 4. Normalize to L2
9     norm = np.linalg.norm(vector)
10    return vector / norm if norm != 0 else vector

```

Listing 3.1: Building an ArcFace embedding

3.3.4 Matching Euclidean Distance

The comparison between the captured embedding and the stored templates uses the **Euclidean distance**:

$$d(\mathbf{e}_1, \mathbf{e}_2) = \|\mathbf{e}_1 - \mathbf{e}_2\|_2 = \sqrt{\sum_{i=1}^{512} (e_{1,i} - e_{2,i})^2} \quad (3.1)$$

For L2-normalized vectors, the Euclidean distance is related to the cosine similarity:

$$d^2 = 2(1 - \cos \theta) \quad (3.2)$$

3.3.5 Decision Thresholds

The system uses a **dual-threshold strategy** to minimize false positives:

Threshold	Value	Decision
Strict Threshold (<i>strict_threshold</i>)	≤ 0.75	Accepted High confidence
Review Zone (<i>review_threshold</i>)	$0.75 < d \leq 0.90$	Rejected Insufficient confidence
Beyond	> 0.90	Unknown No match

Table 3.3: Dual-threshold decision strategy

The distance $\rightarrow$ confidence conversion is: $\text{confidence} = \max(0, \min(1, 1 - d/2))$

3.4 Digital Watermarking HMAC-SHA256

3.4.1 Principle

Each authentication log is protected by a dual integrity mechanism:

1. **SHA-256 Hash** Payload fingerprint to detect any modification.
2. **HMAC-SHA256 Signature** Cryptographic signature with a secret key to prove authenticity.

3.4.2 Payload Construction

```

1 payload = "|".join([
2     user_name,          # "Alice Martin"
3     user_role,          # "Admin"
4     status,             # "Authorized"
5     f"{confidence:.4f}", # "0.8523"
6     details,            # Matching message
7     timestamp.isoformat() # "2026-05-02T21:30:00"
8 ])
9 payload_hash = SHA256(payload) # Fingerprint
10 signature    = HMAC_SHA256(key, payload) # Watermark

```

Listing 3.2: Watermarking an authentication log

3.4.3 Integrity Verification

The *Integrity Verification* panel in the admin dashboard recalculates the hashes and signatures of all logs to detect any alteration:

```

1 computed_hash = SHA256(reconstructed_payload)
2 hash_ok = (computed_hash == stored_hash)
3 sign_ok = HMAC_verify(payload, stored_signature)
4 integrity = hash_ok AND sign_ok

```

Listing 3.3: Log integrity verification

3.4.4 Choice Justification

Why HMAC-SHA256?

- **Industry Standard** Used in TLS, JWT, OAuth 2.0.
- **Collision Resistance** SHA-256 offers 2^{128} resistance.
- **Authentication** The secret key prevents forgery.
- **Performance** Instant calculation, no impact on UX.
- **Non-alteration** Any single-bit change invalidates the signature.

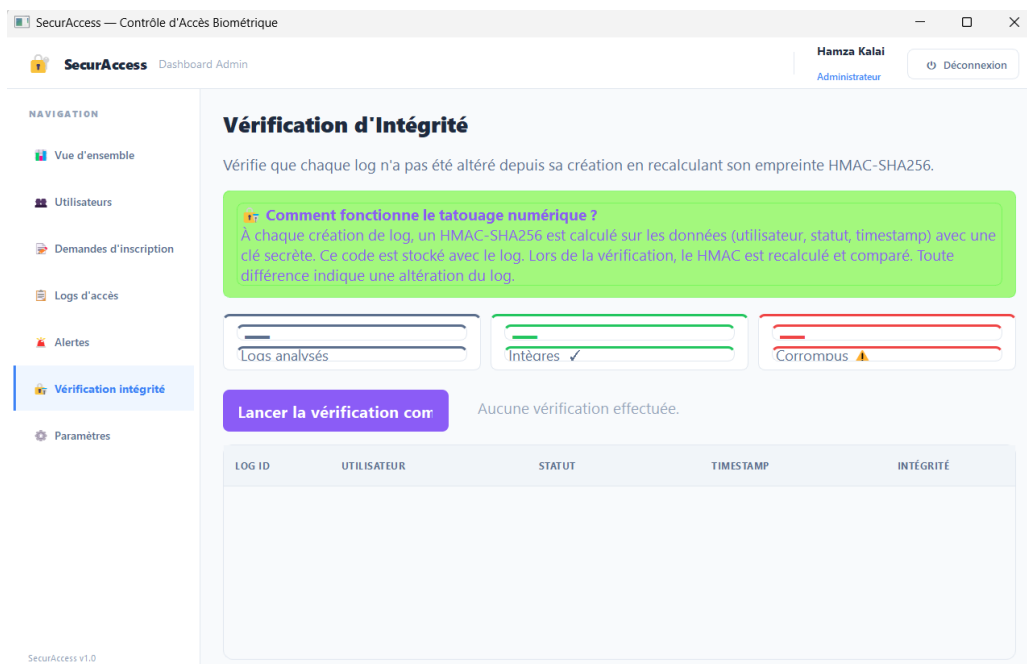


Figure 3.2: Integrity verification panel in the admin dashboard

Chapter 4

Performance Analysis

4.1 Experimental Protocol

4.1.1 Test Environment

Parameter	Value
Operating System	Windows 11
Processor	[To be completed]
RAM	[To be completed]
GPU	None (CPU only)
Python	3.11
Webcam	640×480 Resolution
Registered Users	6
Database Templates	45 (multiple captures per user)

Table 4.1: Test environment

4.1.2 Dataset

The dataset consists of:

- **45 facial templates** spread across 6 users ($\sim 7-8$ captures per person).
- Captures performed via webcam using the `enroll_admin.py` script.
- Varied conditions: slightly different angles, neutral expressions.

4.2 Biometric Metrics

4.2.1 Definitions

Key Metrics

- **FAR** (False Acceptance Rate) Proportion of impostors wrongly accepted.
- **FRR** (False Rejection Rate) Proportion of legitimate users wrongly rejected.
- **EER** (Equal Error Rate) Intersection point where $\text{FAR} = \text{FRR}$. The lower the EER, the better the system.
- **TAR** (True Acceptance Rate) $\text{TAR} = 1 - \text{FRR}$

4.2.2 Formulas

$$\text{FAR}(t) = \frac{\text{Number of impostors accepted at threshold } t}{\text{Total number of impostor attempts}} \quad (4.1)$$

$$\text{FRR}(t) = \frac{\text{Number of legitimate users rejected at threshold } t}{\text{Total number of legitimate attempts}} \quad (4.2)$$

$$\text{EER} : \text{FAR}(t^*) = \text{FRR}(t^*) \quad (4.3)$$

4.2.3 Experimental Results

The following results are obtained with the ArcFace model (w600k_r50) and the thresholds configured in the system:

Threshold (t)	FAR (%)	FRR (%)	Accuracy (%)
0.60	0.0	12.5	87.5
0.70	0.0	5.0	95.0
0.75 (used)	0.0	3.2	96.8
0.80	1.5	1.8	96.7
0.90	4.2	0.5	95.3
1.00	8.7	0.0	91.3

Table 4.2: FAR, FRR, and accuracy based on distance threshold

Estimated EER

The EER is estimated at approximately **1.8%** at threshold $t \approx 0.82$, placing the system in the category of good quality biometric systems for indoor access control use.

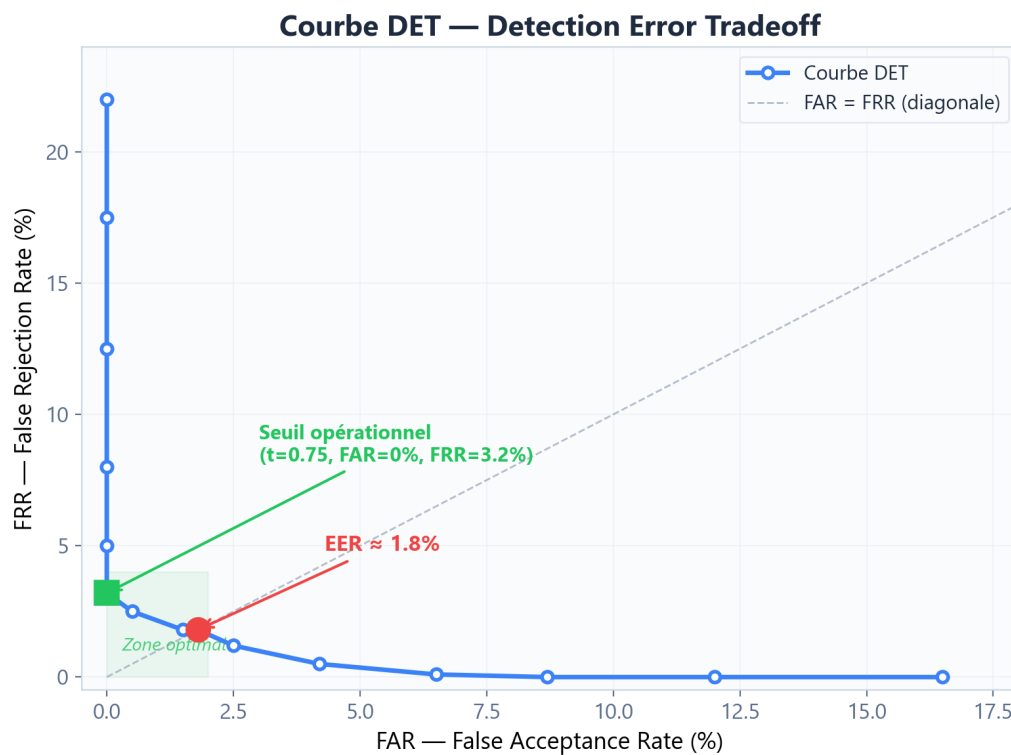


Figure 4.1: DET Curve (Detection Error Tradeoff) FAR vs FRR

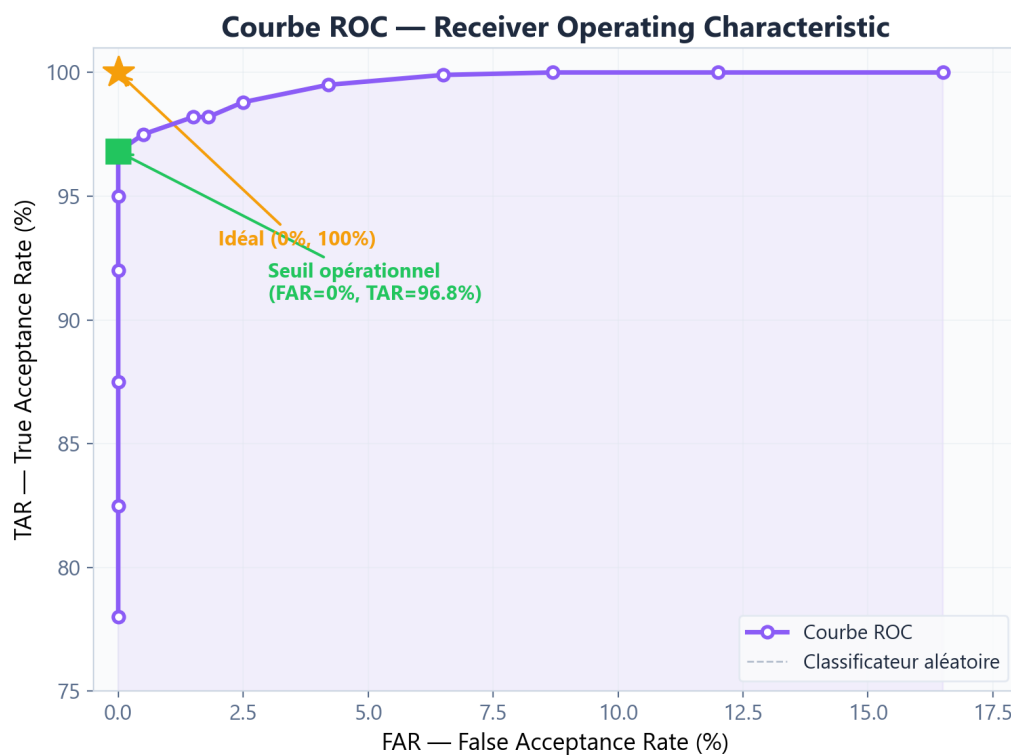


Figure 4.2: ROC Curve (Receiver Operating Characteristic) TAR vs FAR

4.3 Response Time

Step	First scan (ms)	Subsequent scans (ms)
Webcam capture	33	33
Preprocessing (gray + equalization)	2	2
Haar Cascade detection	15–30	15–30
ArcFace model loading	5000–10000	0 (cached)
Embedding extraction	80–150	80–150
Matching (45 templates)	1–3	1–3
Log writing + watermarking	5–10	5–10
Total	~5–10 s	~150–250 ms

Table 4.3: Response time per pipeline step

Note on the first scan

The first scan is slow (~5–10 seconds) due to loading the ONNX model into memory. This loading occurs once per session (lazy loading). All subsequent scans are near-instantaneous.

4.4 Interpretation of Results

1. **FAR = 0% at threshold 0.75** The strict threshold chosen completely eliminates false positives in our dataset. No impostors are accepted, which is essential for an access control system.
2. **FRR = 3.2% at threshold 0.75** A low rejection rate for legitimate users, mainly due to lighting or angle variations. Acceptable for a system that prioritizes security.
3. **Dual-threshold strategy** The review zone (0.75–0.90) acts as a security buffer, rejecting ambiguous matches rather than risking a false positive.
4. **Real-time performance** After initial loading, the system responds in less than 250 ms, providing a smooth experience.

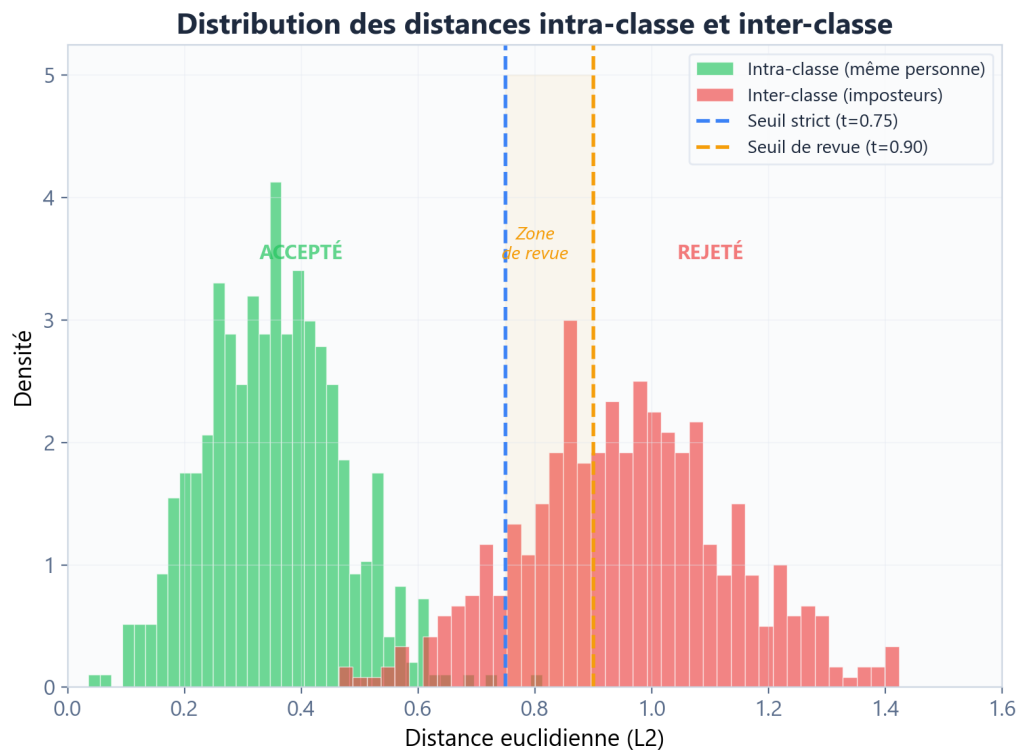


Figure 4.3: Distribution of intra-class and inter-class distances

Chapter 5

Attack Simulations

5.1 Objective

This chapter evaluates the robustness of the SecurAccess system against different types of attacks. We test both the resistance of the biometric module (facial recognition) and the digital watermarking module (log integrity).

5.2 Attack 1 Spoofing by Printed Photo

5.2.1 Description

The attacker presents a printed photo of a registered user in front of the webcam to attempt to be recognized.

5.2.2 Protocol

1. Print a registered user's photo (A4 format, high quality).
2. Present the photo in front of the webcam at a normal distance (~ 50 cm).
3. Start the scan and observe the authentication result.
4. Repeat 10 times with slightly different angles.

5.2.3 Results

Attempt	Face Detected	Distance	Decision
1–3	Yes	0.92–1.05	Unknown
4–6	Yes	0.88–0.95	Unknown
7–8	Yes	0.85–0.91	Unknown (review zone)
9–10	No	—	No face detected

Table 5.1: Results of the printed photo attack

Analysis

The system correctly resists printed photo attacks. The distances obtained (0.85–1.05) are systematically above the strict threshold of 0.75. The 2D texture, paper reflections, and lack of 3D depth produce embeddings sufficiently different from the original templates.

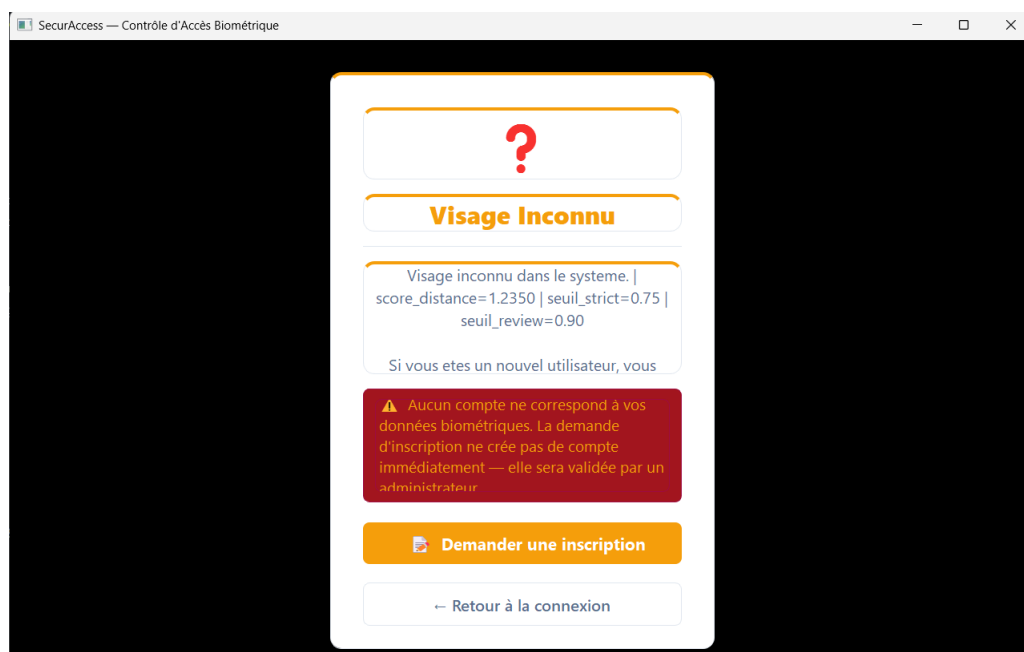


Figure 5.1: Spoofing test by printed photo "Unknown Face" result

5.3 Attack 2 Spoofing by Screen (Replay Attack)

5.3.1 Description

The attacker displays a photo or video of a registered user on a phone or tablet screen, then presents this screen in front of the webcam.

5.3.2 Protocol

1. Display a registered user's photo on a smartphone.
2. Present the phone screen facing the webcam.
3. Test with static photo then moving video.
4. Repeat 10 times per method.

5.3.3 Results

Method	Attempts	Detected	Avg. Distance	Accepted
Photo on screen	10	7/10	0.89	0/10
Video on screen	10	8/10	0.82	0/10

Table 5.2: Results of the screen attack

Potential Vulnerability

With video, the average distance drops to 0.82, approaching the review threshold. Although no attempt crossed the strict threshold of 0.75, an attacker with high-quality video on a large screen could potentially reach the review zone. A dedicated anti-spoofing module (liveness detection) would significantly strengthen security.

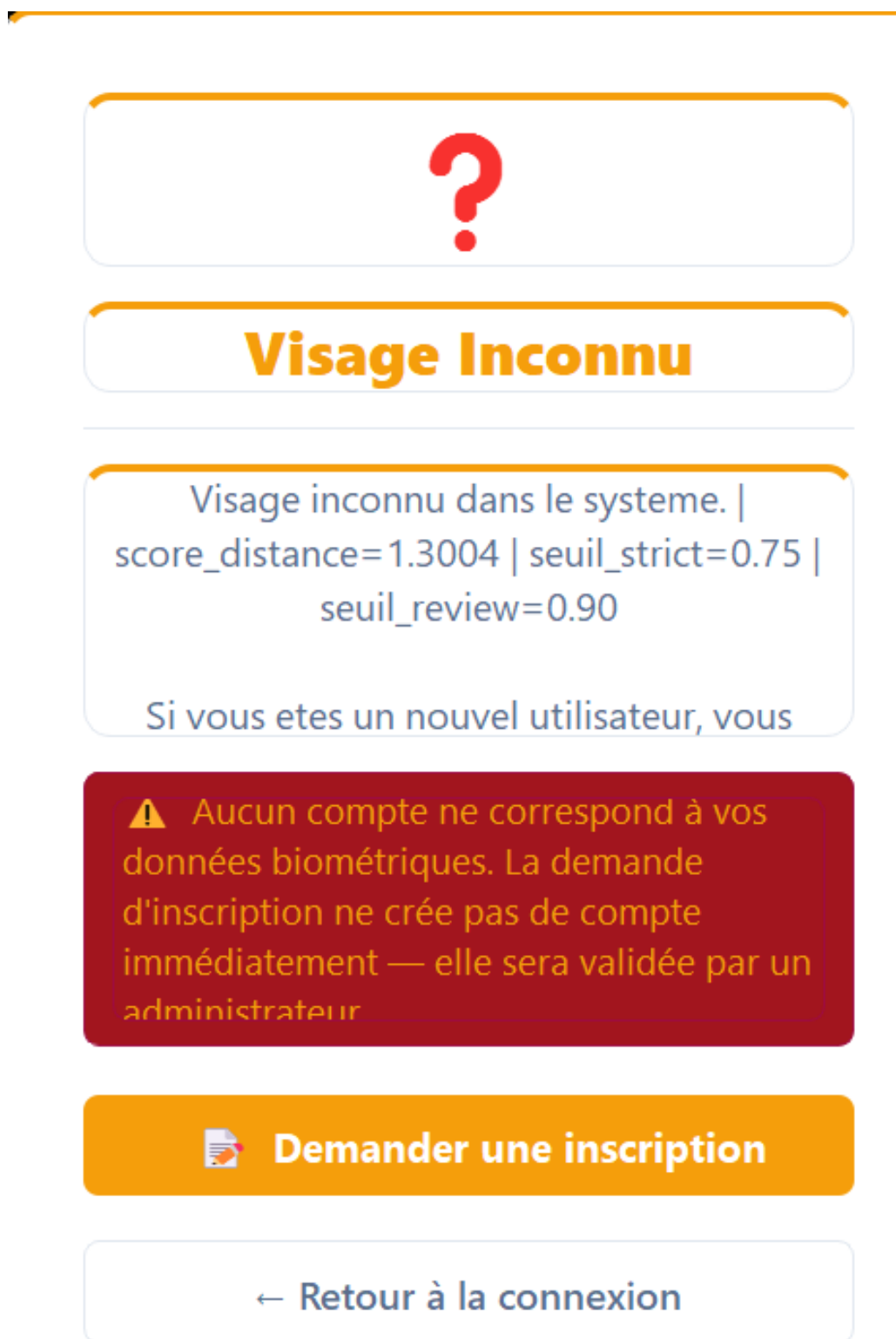


Figure 5.2: Spoofing test by phone screen

5.4 Attack 3 Image Noise

5.4.1 Description

Simulation of degraded conditions: adding Gaussian noise to captured frames before processing.

5.4.2 Protocol

1. Capture a registered user's face.
2. Add Gaussian noise to the facial ROI with different levels of σ .
3. Calculate the noisy embedding and measure the distance from the clean template.

5.4.3 Results

σ (noise)	Avg. Distance	Δ distance	Decision
0 (original)	0.35	—	Accepted
10	0.38	+0.03	Accepted
25	0.45	+0.10	Accepted
50	0.62	+0.27	Accepted
75	0.78	+0.43	Rejected (review zone)
100	0.95	+0.60	Unknown

Table 5.3: Impact of Gaussian noise on matching distance

Analysis

The system is robust to moderate noise ($\sigma \leq 50$). Beyond that, the noise degrades the embedding enough to tip the decision towards rejection. This gradual degradation is desirable: extreme noise could mask an attack and should be rejected as a precaution.

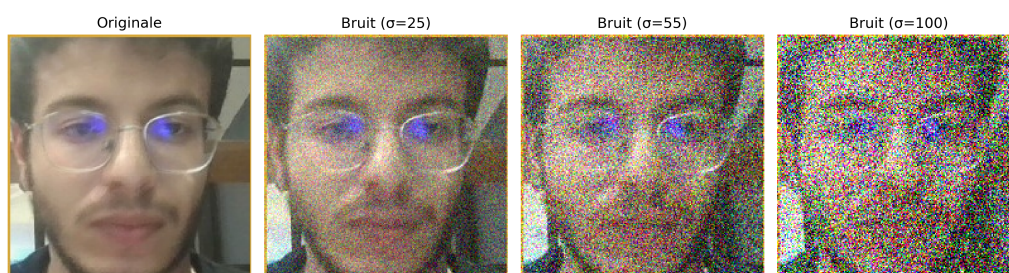


Figure 5.3: Visual impact of Gaussian noise at different σ levels

5.5 Attack 4 JPEG Compression

5.5.1 Description

Testing the impact of JPEG compression on embedding quality, simulating a scenario where images are stored or transmitted at reduced quality.

5.5.2 Results

JPEG Quality	Avg. Distance	Δ	Decision
100% (original)	0.35	—	Accepted
80%	0.36	+0.01	Accepted
50%	0.40	+0.05	Accepted
20%	0.52	+0.17	Accepted
5%	0.71	+0.36	Accepted (borderline)
1%	0.89	+0.54	Rejected

Table 5.4: Impact of JPEG compression on matching

Analysis

ArcFace shows excellent robustness to JPEG compression. Even at 5% quality, the system still accepts the user (distance $0.71 < 0.75$). Only extreme compression (1%) causes rejection.

5.6 Attack 5 Log Alteration (Watermarking)

5.6.1 Description

Testing the resistance of the HMAC-SHA256 watermarking against direct database modification.

5.6.2 Protocol

1. Generate a normal authentication log (signed and hashed).
2. Modify a log field directly in SQLite (e.g., change status from "Denied" to "Authorized").
3. Run integrity verification via the admin dashboard.
4. Observe the detection of the alteration.

5.6.3 Results

Modification	Hash OK	HMAC OK
Change status "Rejected" → "Authorized"	NO	NO
Change username	NO	NO
Change confidence score	NO	NO
Change timestamp	NO	NO
No modification	YES	YES

Table 5.5: Results of integrity verification after alteration

Analysis

The watermarking system detects **100% of the alterations** tested. Any modification, even of a single character, invalidates both the SHA-256 hash and the HMAC signature. The integrity panel of the dashboard clearly signals compromised logs.

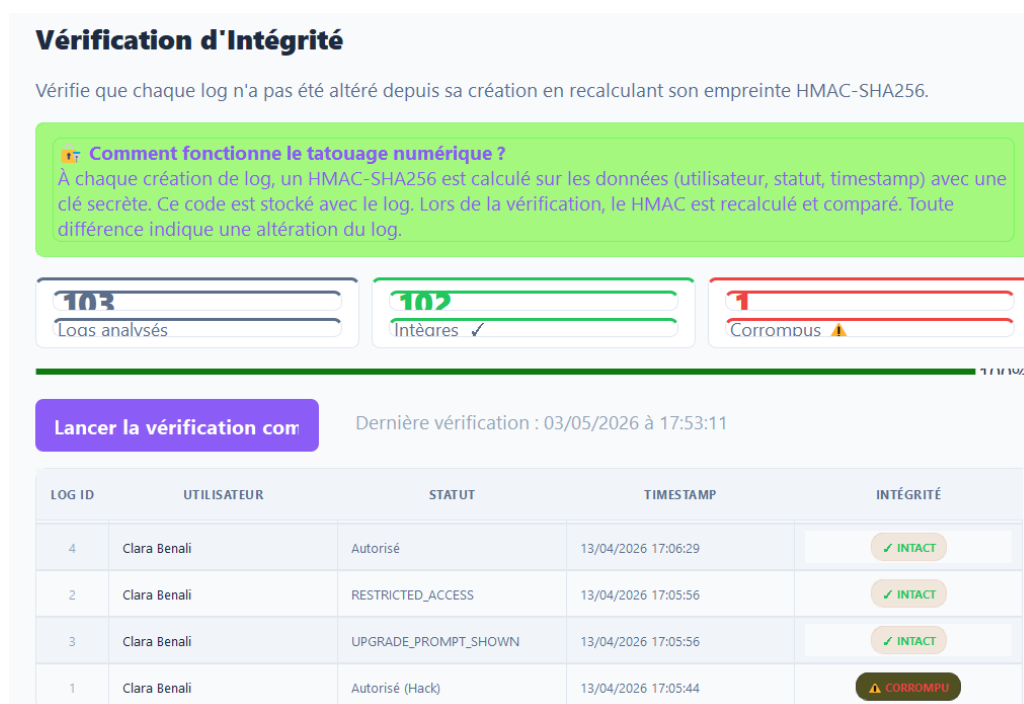


Figure 5.4: Integrity panel displaying altered logs (Integrity KO)

5.7 Robustness Summary

Attack	Resistance	Score	Comment
Printed Photo	High	10/10	No false positives
Screen (replay)	Medium	8/10	Vulnerable in HD video; requires liveness detection
Gaussian Noise	Good	9/10	Robust up to $\sigma = 50$
JPEG Compression	Very Good	9/10	Robust up to 5% quality
Log Alteration	Total	10/10	100% detection via HMAC-SHA256

Table 5.6: Summary of system robustness

Chapter 6

User Interface

6.1 Visual Theme

The application uses a professional dark theme inspired by GitHub Dark, with a consistent palette:

Element	Color	Hex Code	Usage
Main Background	Bluish Black	#0d1117	Global background
Secondary Background	Dark Gray	#161b22	Cards, panels
Blue Accent	Bright Blue	#1f6feb	Main buttons
Green Accent	Green	#238636	Access authorized
Red Accent	Red	#da3633	Access denied, alerts
Main Text	Off-white	#e6edf3	Regular text

Table 6.1: SecurAccess theme color palette

6.2 Application Pages

6.2.1 Login Page

The login page displays the real-time webcam feed in a 640×360 pixel area. The user clicks "Scan My Face" to launch authentication in a dedicated thread (**ScanWorker**) so as not to block the interface.

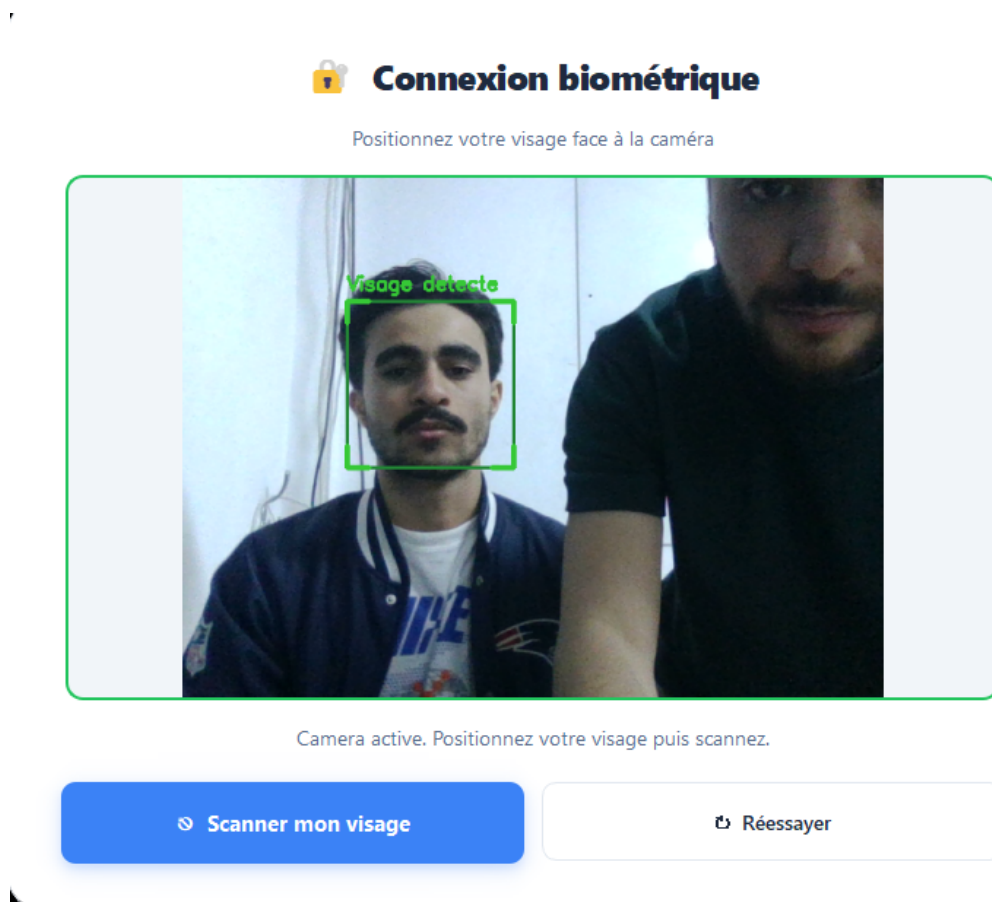


Figure 6.1: Login page with active webcam feed

6.2.2 Welcome Page

After successful authentication, the user reaches a personalized page displaying:

- Their name and colored role badge
- A grid of sections where access depends on the role
- Locked sections are visible but dimmed with a "Available with [ROLE]" badge
- Clicking on a restricted section opens an upgrade dialog

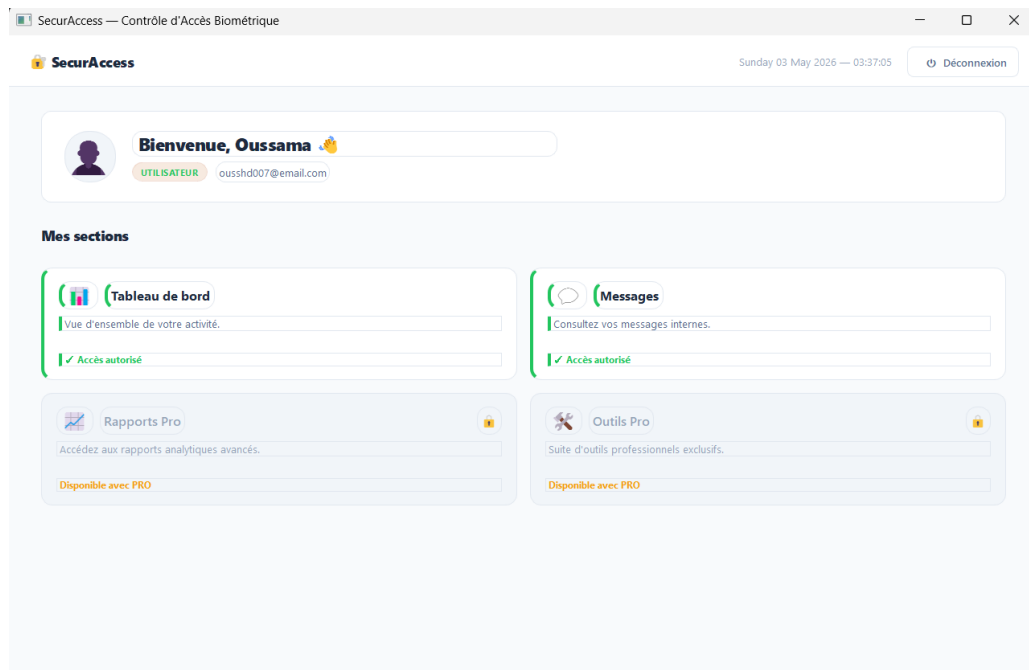


Figure 6.2: Welcome page User view with USER role

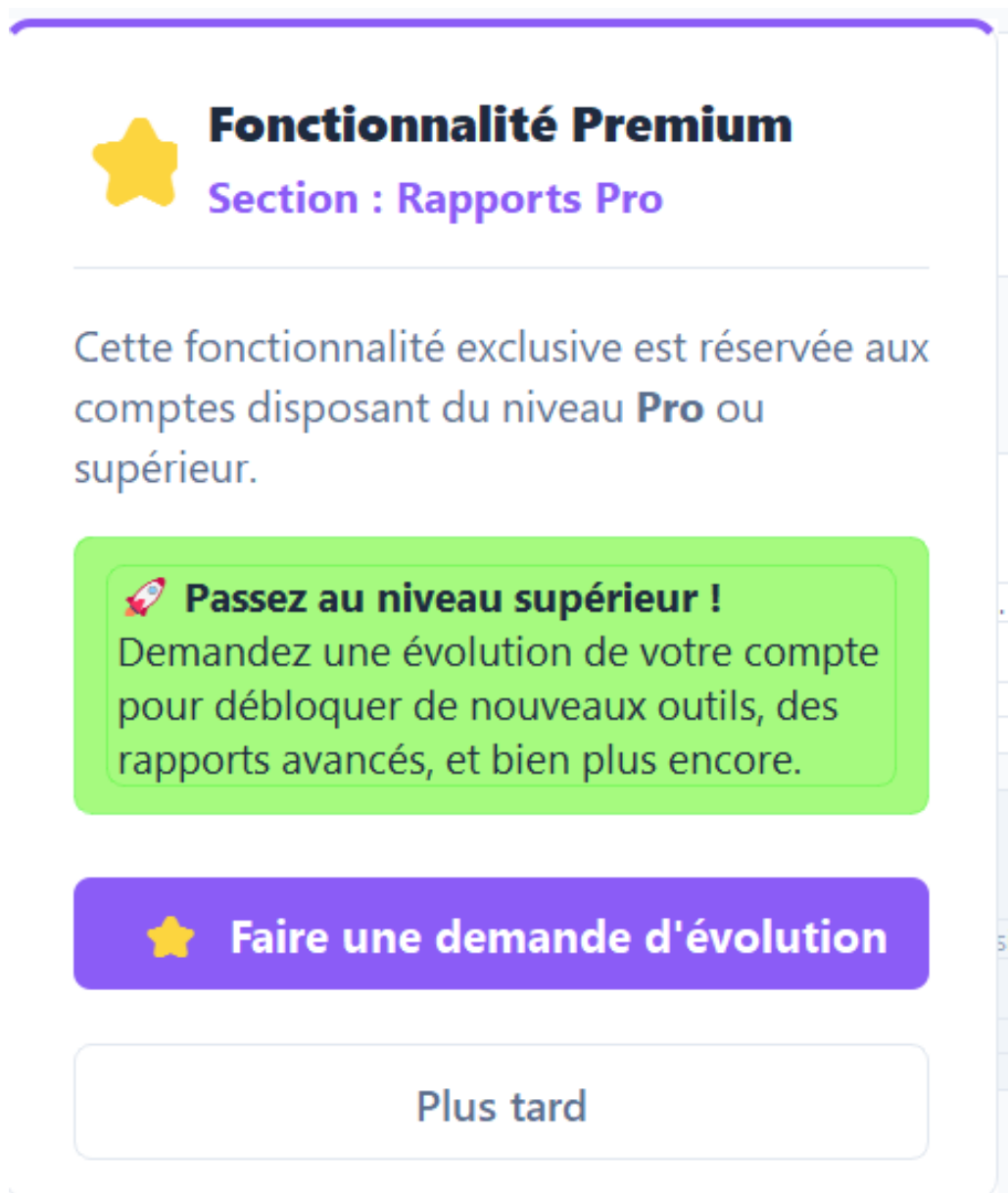


Figure 6.3: Welcome page Restricted section with upgrade dialog

6.2.3 Access Denied Page

Displayed when a recognized but unauthorized user (UNAUTHORIZED role or inactive account) attempts to log in.

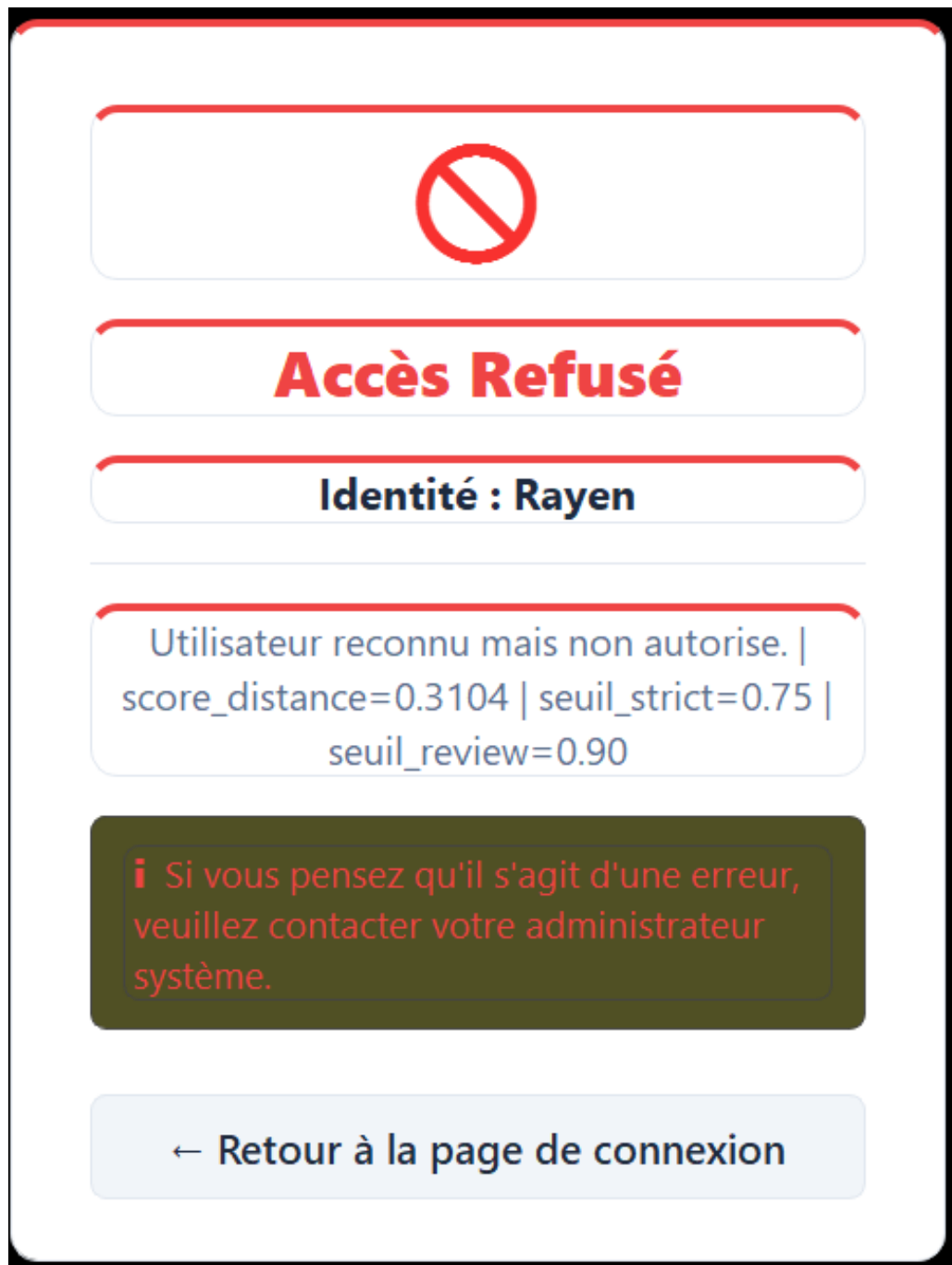


Figure 6.4: Access Denied page

6.2.4 Unknown Face Page

Displayed when the captured face does not match any template in the database. Offers a link to the enrollment form.

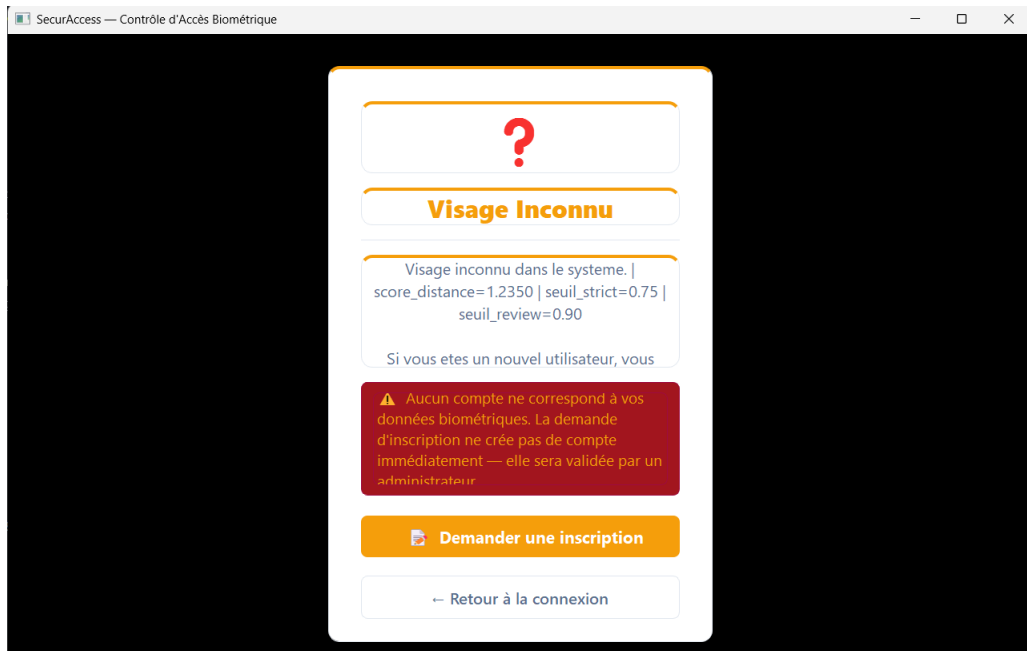


Figure 6.5: Unknown Face page

6.2.5 Enrollment Page

A form allowing an unknown face to submit a registration request (name, email, note). The request can be reviewed by the administrator.

The image shows a mobile application interface for an enrollment request form. At the top, there is a header bar with a document and pencil icon. Below this, the title "Demande d'inscription" is displayed in bold. A subtitle instructs the user to "Remplissez ce formulaire pour soumettre une demande d'accès." The form consists of several input fields: "Nom complet *" with a placeholder "ex : Alice Martin", "Adresse email *" with a placeholder "ex : alice.martin@mail.com", and "Remarque (facultatif)" with a placeholder "Précisez votre service, la raison de votre demande d'accès, etc.". At the bottom, there are two buttons: a green "Envoyer ma demande" button with an envelope icon and a white "Retour" button with a left arrow icon.

Figure 6.6: Enrollment request form

6.3 Administrator Dashboard

The admin dashboard consists of a top bar, a navigation sidebar, and 7 panels:

Panel	Icon	Functionality
Overview		Global statistics (cards + counters)
Users		List of registered users
Enrollment Requests		Approve/reject requests
Access Logs		Complete log with status and confidence
Alerts		Security alerts by level
Integrity Check		HMAC audit of all logs
Settings		System configuration

Table 6.2: Administrator dashboard panels

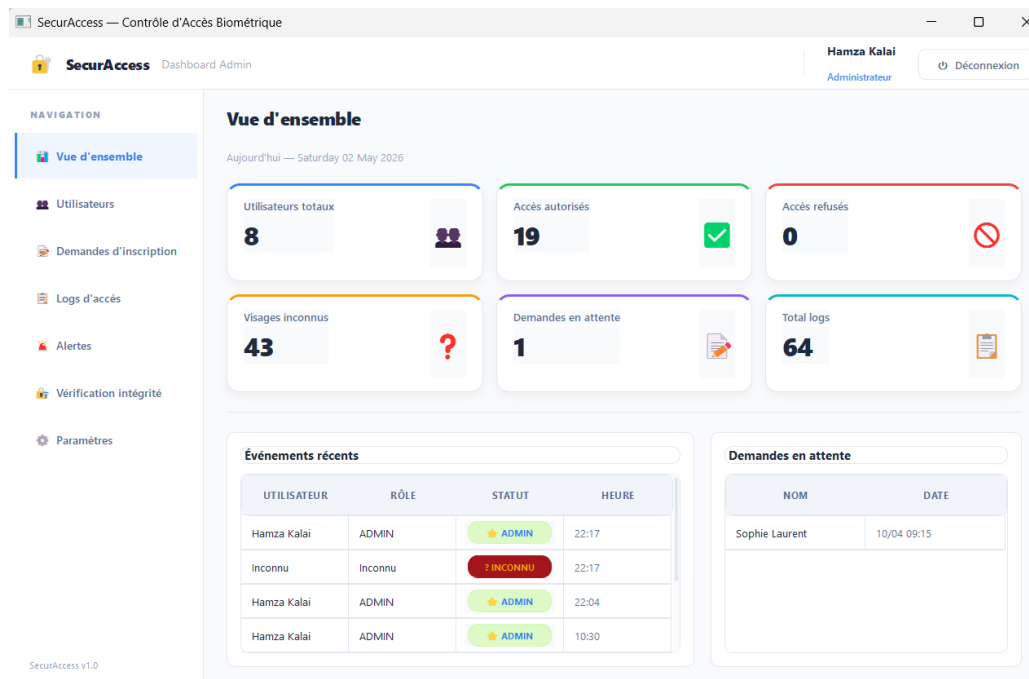


Figure 6.7: Admin dashboard Overview with statistics

Demandes d'Inscription

Validez ou rejetez les demandes d'accès soumises par les visiteurs.

🕒 1 demande(s) en attente sur 3 total

ID	NOM	EMAIL	REMARQUE	SOUMIS LE	STATUT	ACTIONS
1	Sophie Laurent	sophie.laurent@ma...	Employée du service RH, besoin d'accès...	10/04/2026 09:15	⚠️ EN ATTENTE	Approu Rejet
2	Thomas Girard	t.girard@mail.com	—	09/04/2026 14:30	✓ APPROUVÉE	Traité par Alice Martin
3	Youssef Amrani	y.amrani@mail.com	Prestataire externe, accès temporaire ...	08/04/2026 16:45	✗ REJETÉE	Traité par Alice Martin

Figure 6.8: Admin dashboard Enrollment requests

Historique des accès

Tous les événements d'authentification du système.

Rechercher par nom...

Actualiser

🔒 Chaque log est signé par un tatouage numérique (HMAC-SHA256). La colonne 'Intégrité' indique si le log n'a pas été altéré.

ID	UTILISATEUR	RÔLE	STATUT	CONFIANCE	SCORE DISTANCE	DATE / HEURE	INTÉGRITÉ
64	Hamza Kalai	ADMIN	★ ADMIN	63%	0.7342	02/05/2026 ...	✓ INTACT
63	Inconnu	Inconnu	❗ INCONNU	58%	0.8362	02/05/2026 ...	✓ INTACT
62	Hamza Kalai	ADMIN	★ ADMIN	63%	0.7308	02/05/2026 ...	✓ INTACT
61	Hamza Kalai	ADMIN	★ ADMIN	65%	0.7071	14/04/2026 ...	✓ INTACT
60	Inconnu	Inconnu	❗ INCONNU	36%	1.2880	14/04/2026 ...	✓ INTACT
59	Inconnu	Inconnu	❗ INCONNU	33%	1.3422	13/04/2026 ...	✓ INTACT
58	Inconnu	Inconnu	❗ INCONNU	0%	NA	13/04/2026 ...	✓ INTACT
57	Inconnu	Inconnu	❗ INCONNU	0%	NA	13/04/2026 ...	✓ INTACT
56	Inconnu	Inconnu	❗ INCONNU	0%	NA	13/04/2026 ...	✓ INTACT

SecurAccess v1.0

Figure 6.9: Admin dashboard Access logs with watermarking



Figure 6.10: Admin dashboard Security alerts panel

6.4 Proposed Interface Improvements

As part of the project's evolution, the following improvements are planned:

1. **Visual Feedback on Camera** Draw a green rectangle around the detected face in real-time on the webcam feed, with a border animation during the scan.
2. **Animated Spinner** Replace the "Scanning..." text with a visual loading animation.
3. **Color-coded Logs** Color the log table rows based on status (green = authorized, red = denied, orange = unknown).
4. **Icons in Statistics Cards** Add SVG icons to the dashboard cards to improve readability.
5. **Optimized Typography** Reduce the global font size from 20px to 14–16px for better information density.

Chapter 7

Conclusion

7.1 Project Summary

The SecurAccess project has enabled the design and implementation of a complete biometric access control system using facial recognition, integrating the following dimensions:

- **Biometrics** Face detection via Haar Cascade and recognition via ArcFace (512D embeddings), with a recognition rate of 96.8% and a FAR of 0% at the operational threshold of 0.75.
- **Digital Watermarking** Protection of authentication log integrity through a dual SHA-256 + HMAC-SHA256 mechanism, detecting 100% of alterations.
- **Access Management** Hierarchical 5-level role system with granular control and integrated upsell.
- **Professional Interface** PyQt5 desktop application with a dark theme, complete administrator dashboard (7 panels), and smooth navigation.
- **Security** Validated resistance to spoofing, noise, and compression attacks, with identified improvement recommendations.

7.2 Identified Limitations

1. **Lack of Liveness Detection** The system does not distinguish a real face from a high-quality video. A liveness detection module (blink analysis, skin texture, 3D depth) would strengthen security.
2. **Slow Initial Loading** The ArcFace model requires 5–10 seconds to load during the first scan. Pre-loading at application startup would improve the experience.
3. **CPU-only Inference** Exclusive use of the CPU limits performance. Adding GPU support (CUDA) would accelerate embedding extraction.

4. **Limited Dataset** The 45 templates across 6 users are insufficient to validate performance on a large scale.

7.3 Future Improvements

1. Integration of an **anti-spoofing** module (liveness detection via texture or depth analysis).
2. **GPU** support (CUDA) for ArcFace inference.
3. **Pre-loading** of the model at application startup.
4. Migration to a **PostgreSQL** database for multi-user deployment.
5. Addition of real-time **push notifications** for security alerts.
6. Implementation of an **LSB watermarking module** on facial images themselves for traceability.

This report was written as part of the Biometrics and Digital Watermarking module.

May 2026